

Experimental Setup

This chapter describes the experimental environment used to evaluate the proposed side-channel Trojan-detection methodology: the benchmark circuit and its FPGA mapping, the software toolchain, the Trojan and trigger model, the test-generation configuration, the physical-region targeting parameters, and the evaluation metrics. All experiments were run with a single, consistent configuration so that the reported results are reproducible.

1. Benchmark circuit and FPGA mapping

The evaluation uses **s38417**, one of the larger sequential circuits of the ISCAS'89 benchmark suite, which is widely adopted in the hardware-Trojan and test-generation literature. The circuit was synthesised to a Xilinx LUT/flip-flop netlist and then translated to a gate-level .bench representation suitable for simulation: each LUT's initialisation is expanded into primitive AND/OR/NOT/BUF gates, and each FDRE flip-flop is emitted as a scan-style $Q = DFF(D)$ element. After parsing, the circuit has the following dimensions:

Property	Value
Benchmark	s38417 (ISCAS'89, sequential)
Source netlist	Xilinx LUT/FDRE Verilog -> .bench
Primary inputs (incl. scan)	1,559
Primary outputs (incl. scan)	1,633
Gates	43,001
Flip-flops (scan DFFs)	1,527
Simulation mode	Full-scan, levelized vectorized engine

Simulation is performed in **full-scan** mode: flip-flop outputs are treated as pseudo-primary inputs and flip-flop inputs as pseudo-primary outputs, removing sequential feedback so the combinational core can be evaluated directly. The simulator is a levelized, vectorized engine that evaluates many test vectors in a single batched pass, which makes the repeated simulations required by test generation and evaluation tractable on a circuit of this size.

For the physical-region experiments, the synthesised design was placed and routed in **Xilinx Vivado**, and the slice coordinate (x, y) of every placed leaf cell was exported to a CSV. The placed design occupies a fabric area of approximately 60×134 slices, which defines the coordinate grid on which physical regions are formed.

2. Software toolchain

The detection methodology is implemented in Python as a modular pipeline. The main components are:

- **Circuit parser / simulator** — parses the .bench netlist and provides the levelized vectorized simulation used throughout.
- **Rare-node finder** — estimates each node's signal probability by Monte-Carlo simulation and flags low-controllability (rare) nodes, excluding synthetic LUT-artefact nets.

- **MERS test generator** — produces Dataset B by Multiple Excitation of Rare Switching, with the MERS-h (Hamming-distance) and MERS-s (simulation-aware) reordering variants.
- **Dataset-A generator** — produces the paired control test set that suppresses suspect-cone switching while matching Dataset B's background and input/output activity.
- **Region builder** — forms logical (fan-in cone) or physical (Vivado-placement) regions over the rare nodes and selects the top-K.
- **Evaluator and reporter** — scores test sets against a Trojan population and exports Excel/PDF reports.

Numerical work uses NumPy; reports use openpyxl and ReportLab. Placement is exported from Vivado through a Tcl script. All experiments were executed on a standard desktop CPU; no GPU or specialised hardware is required.

3. Trojan and trigger model

Trojans are modelled following the established rare-triggered template used in the MERS literature. A Trojan's *trigger* is a combinational AND of several rare nodes, each required at its rare value; the trigger fires only when all of them are simultaneously satisfied, which is, by construction, a low-probability event. The *payload* is taken as the XOR of an internal net with the trigger signal, so that under a dormant trigger the payload follows the normal net and the Trojan stays hidden.

Two trigger widths were evaluated to test sensitivity to trigger complexity:

Parameter	Value
Trigger type	AND of rare nodes at their rare values
Trigger widths	4-input and 8-input
Payload	internal_net XOR trigger (dormant by default)
Trojan population size	100 random Trojans per width
Rare-value polarity	per-node, from Monte-Carlo estimate

For the differential A/B experiment a specific 4-node trigger was used as the suspect target, with the four trigger nets identified by name (`Q_i_2__292_n_0`, `Q_i_2__363_n_0`, `Q_i_4__103_n_0`, `Q_i_6__5_n_0`) and their activating values, so that suppression could be measured directly on the actual trigger.

4. Test-generation configuration

Rare nodes are identified by Monte-Carlo simulation. The detection sample size is decoupled from the generation pool size: a large sample is used for an accurate rareness estimate, while a smaller pool of candidate vectors is used to build the test set. A node is declared rare if its probability of reaching its less-likely value is below the threshold.

Parameter	Symbol / flag	Value
Rare-node threshold	--threshold	0.1
Rare-detection vectors	--rare-vectors	up to 100,000
Generation pool size	--vectors	1,000 - 10,000
Per-node activation count	--N	100
Mutation mode	--mutation-mode	fast
Mutation rounds	--mutation-rounds	12
MERS-s simulation window	--C	5
Random baseline	--	equal-size random test set

The detection-sample size was raised to 100,000 vectors for the rare-node and region experiments to obtain a clean, well-converged rare-node set (at this sample size no nodes are mis-classified as stuck, whereas small samples leave borderline nodes incorrectly excluded). MERS then builds Dataset B from the candidate pool using $N = 100$ required activations per rare node and 12 mutation rounds in fast mode, followed by the MERS-h and MERS-s reordering passes. The simulation-aware MERS-s variant was used as Dataset B for the differential experiment.

4.1 Paired control set (Dataset A)

Dataset A is generated to pair vector-for-vector with Dataset B. For each pair it minimises switching of the suspect cone while matching B's per-pair background switching, input Hamming distance/weight, and output Hamming distance/weight. By default the suspect cone is seeded from all rare nodes; when region targeting is enabled, Dataset A automatically reuses the same region selection as Dataset B so that the two sets target an identical node set.

5. Physical-region targeting parameters

Physical regions group rare nodes by Euclidean proximity on the Vivado slice grid. Two rare nodes join the same region if their slice-distance is within the clustering radius; regions are grown as compact, size-capped pockets and ranked by rare-node count and density. The configuration used is:

Parameter	Flag	Value
Region type	--placement	physical (Vivado placement)
Clustering radius	--phys-radius	6 slices
Clustering mode	--region-mode	compact
Max region size	--region-max-size	40 rare nodes

Regions selected	--region-top-k	5 (top-K densest)
------------------	----------------	-------------------

The radius of 6 slices was selected from a radius sweep as the smallest value at which the region population stabilises (the knee of the singleton-count curve) for the s38417 placement. At this setting the 302 rare nodes formed 18 physical regions; the top-5 regions, containing 186 rare nodes, were targeted — a 38% reduction in the set MERS must excite.

6. Evaluation metrics

Test sets are assessed with two complementary families of metrics.

6.1 Detection power of Dataset B

The raw Trojan-detection power of a test set is measured by the **Side-Channel Sensitivity (SCS)**, reported as MaxRelSw — the maximum relative switching the test set induces in a Trojan's trigger logic. A higher SCS means the test set drives the hidden trigger harder and makes it more visible in a side-channel trace. SCS is computed over the 100-Trojan populations for both the 4- and 8-input trigger widths, and each MERS variant is compared against the random baseline.

6.2 Differential A/B quality

The paired control is assessed by how well it isolates the trigger while matching everything else. The reported quantities are:

- **Per-trigger P(at activating value)** — how often each trigger node reaches its firing value (lower for A is better suppression).
- **Per-trigger toggle rate** — switches per vector pair on each trigger node (the side-channel-relevant activity).
- **Trigger-cone switching per pair** — the aggregate trigger switching, reported as A relative to B.
- **Matched-control metrics** — background switching, input Hamming distance/weight, and output Hamming distance/weight, which must stay matched between A and B for the differential to be valid.

Switching is measured between consecutive vectors, so a test set of n vectors yields $n-1$ transition pairs; all per-pair metrics are averaged over these transitions.

Reproducibility. Test generation is deterministic for a fixed random seed, and every stage is checkpointed, so a reported test set and its metrics can be regenerated exactly from the same benchmark and configuration. Rare-node detection, region selection, and both test sets are cached and keyed by their parameters.